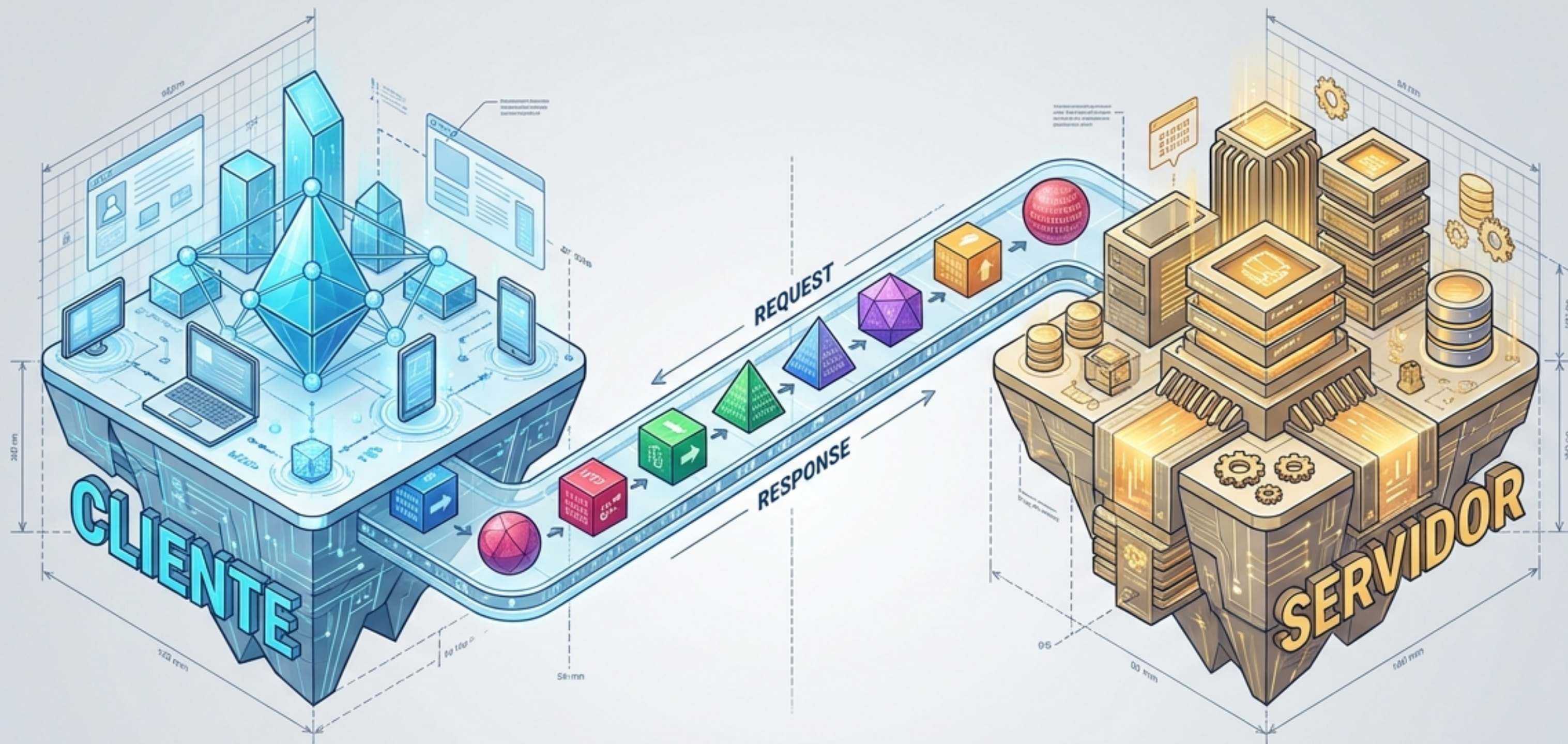


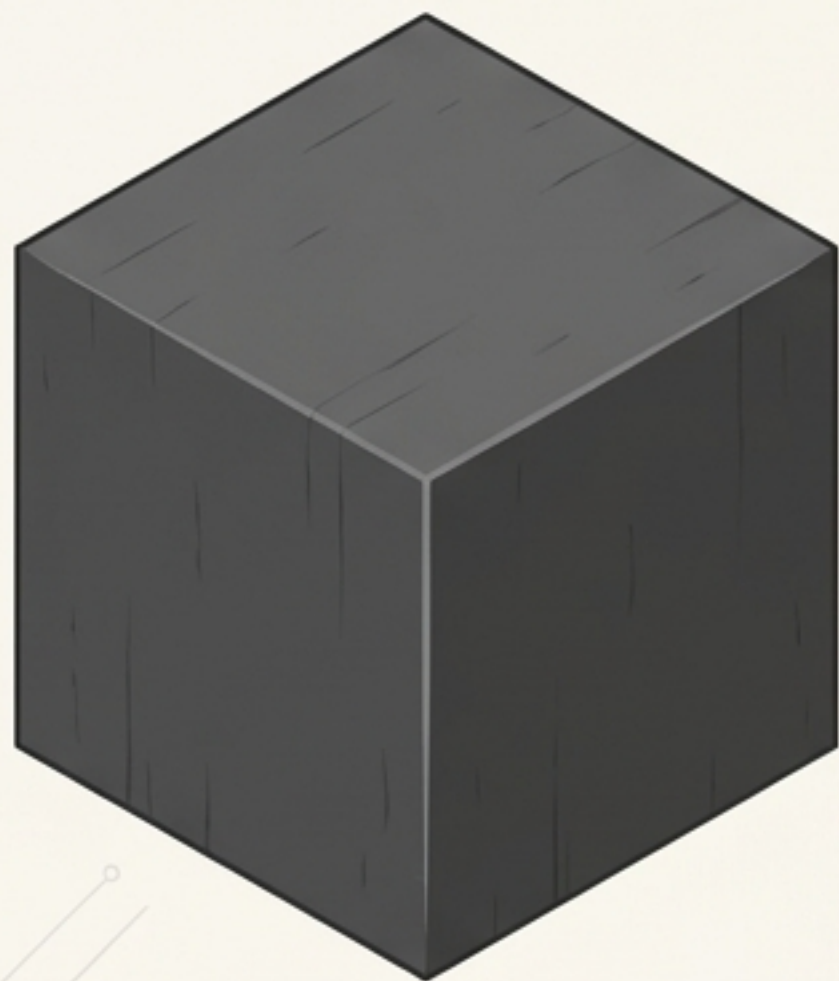


# A Logística da Web: Desconstruindo a Arquitetura Cliente-Servidor

Um guia visual sobre como a web funciona, do HTTP tradicional aos pipelines contínuos do WebSocket.

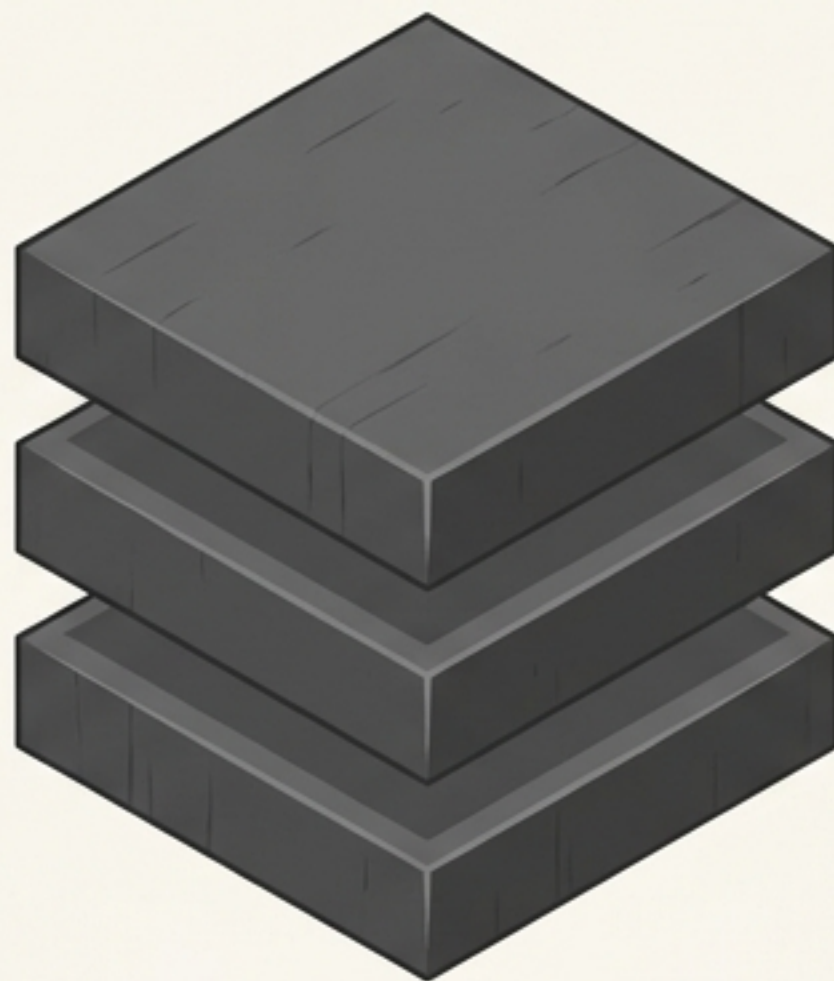


# A fragmentação gera especialização



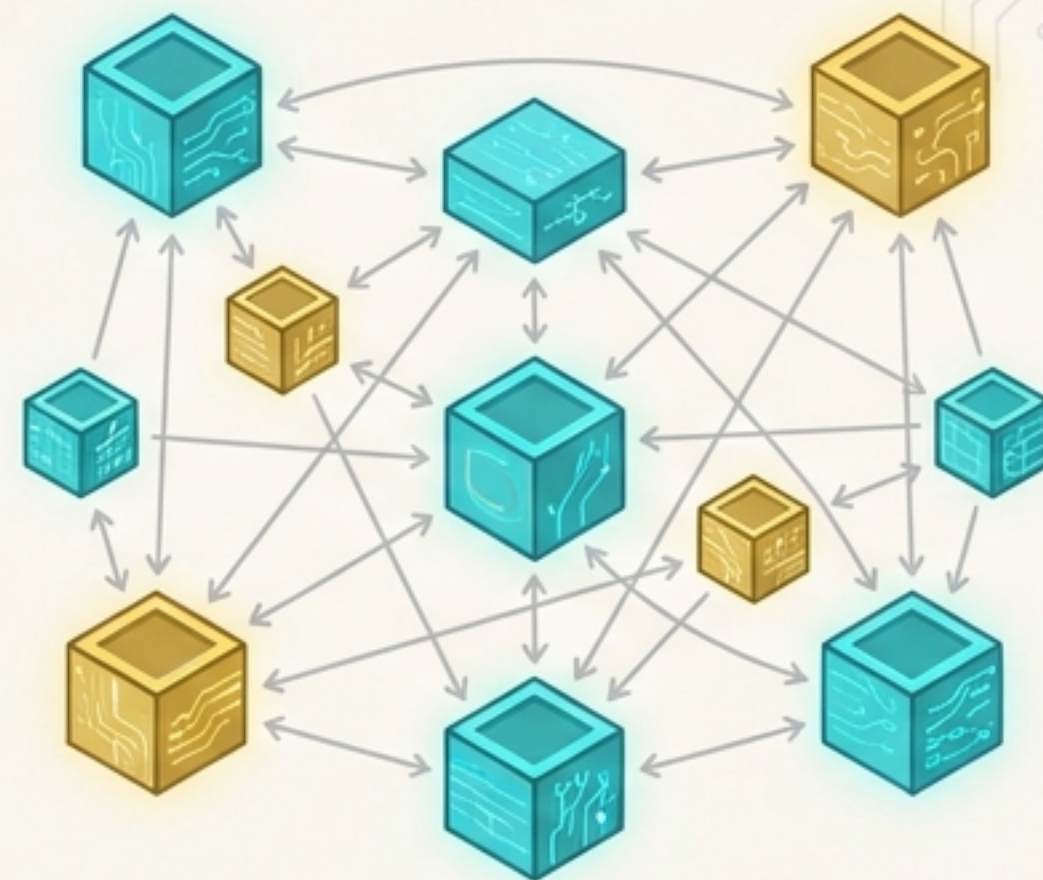
## Monolítico

Sistemas antigos concentravam tudo em uma única máquina.



## Camadas

O software é dividido, onde cada camada tem uma responsabilidade específica e se comunica apenas com camadas adjacentes para modularidade.



## Sistemas Distribuídos

A evolução natural das camadas. Nas palavras de Ian Sommerville: Uma coleção de computadores independentes que aparece para o usuário como um único sistema coerente.



# Os protagonistas da comunicação



## O Cliente

- O Cliente (Solicitante)
- Entidade que solicita recursos ou serviços.
- Fornece a interface com o usuário, envia os **pedidos de acesso e processa os dados recebidos localmente.**

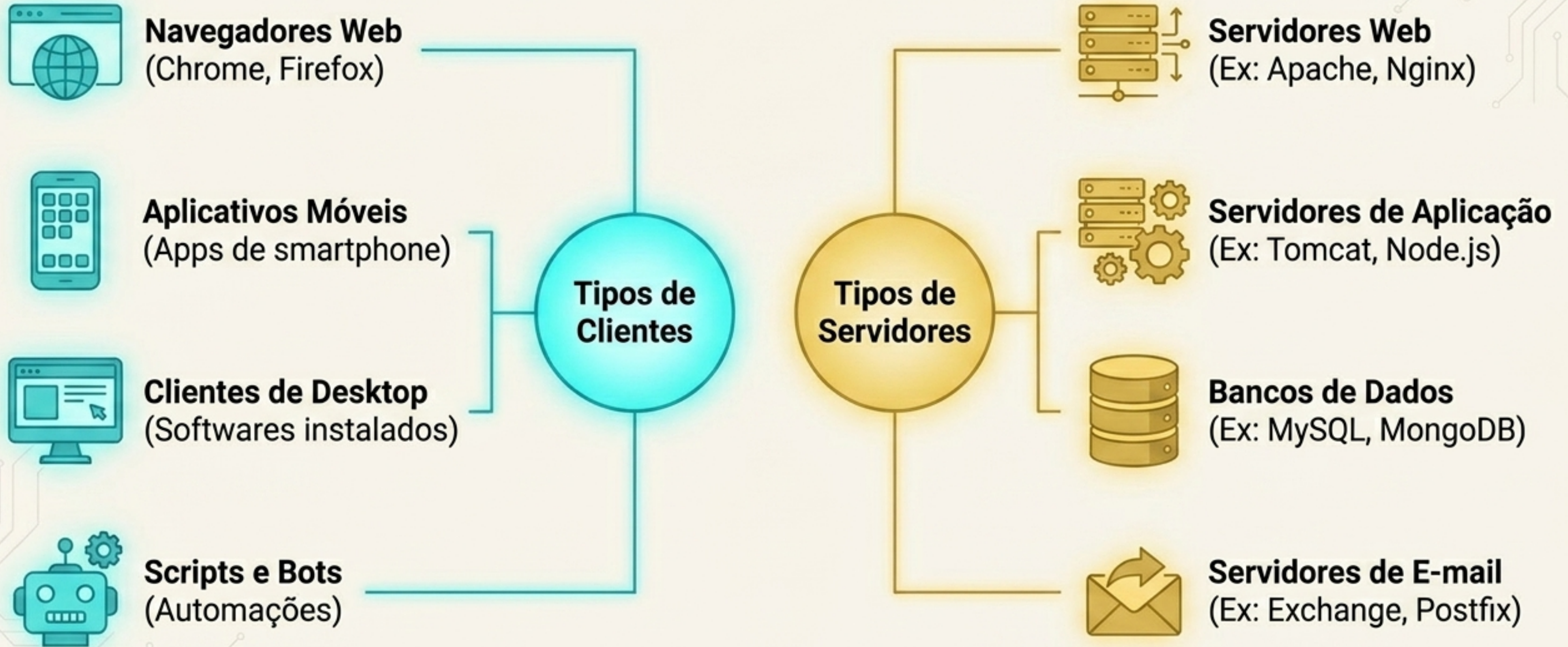


## O Servidor

- O Servidor (Provedor)
- Entidade que gerencia os recursos.
- Mantém os bancos de dados, executa a **lógica de negócios pesada (processamento de solicitações)** e envia as respostas processadas de volta à rede.

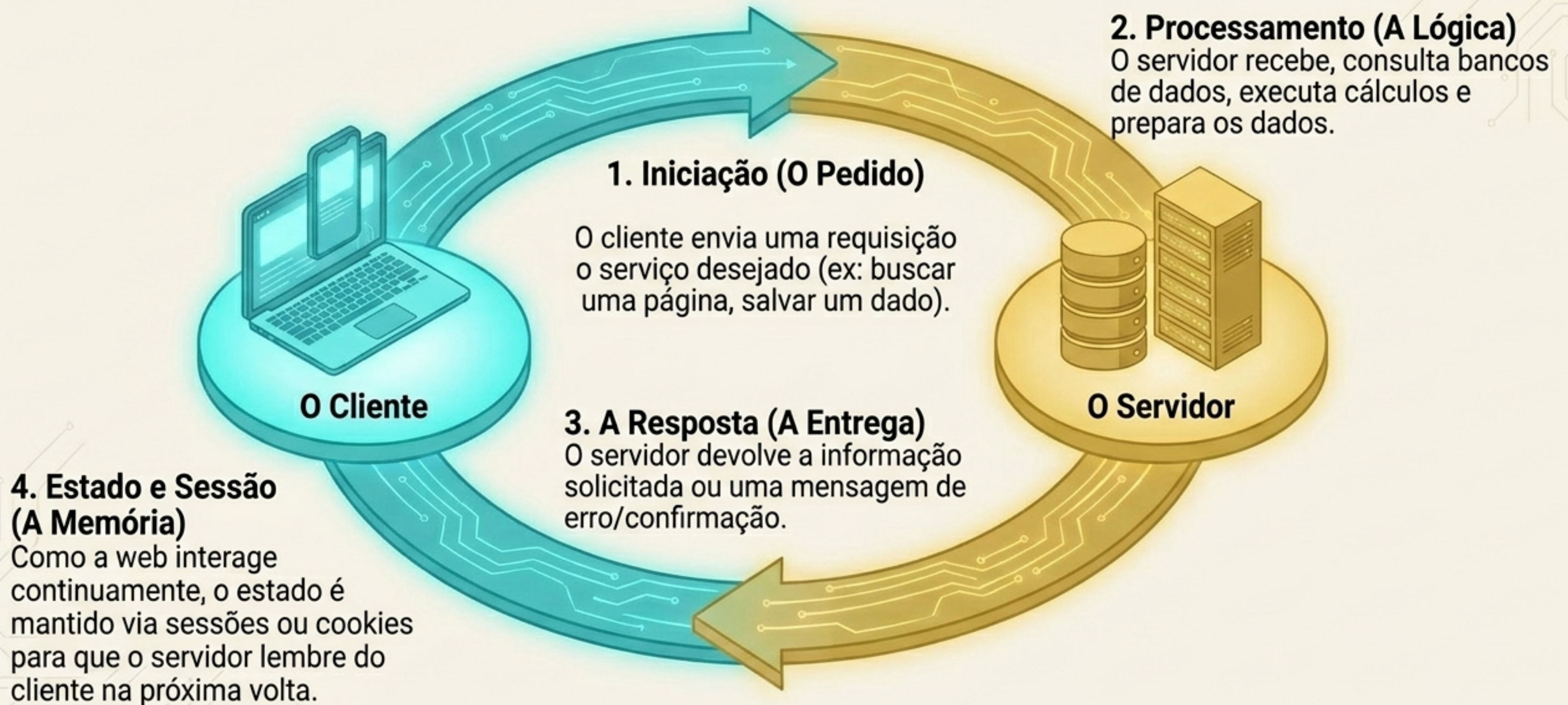


# Um ecossistema de especialistas





# O ciclo vital da web: Requisição e Resposta





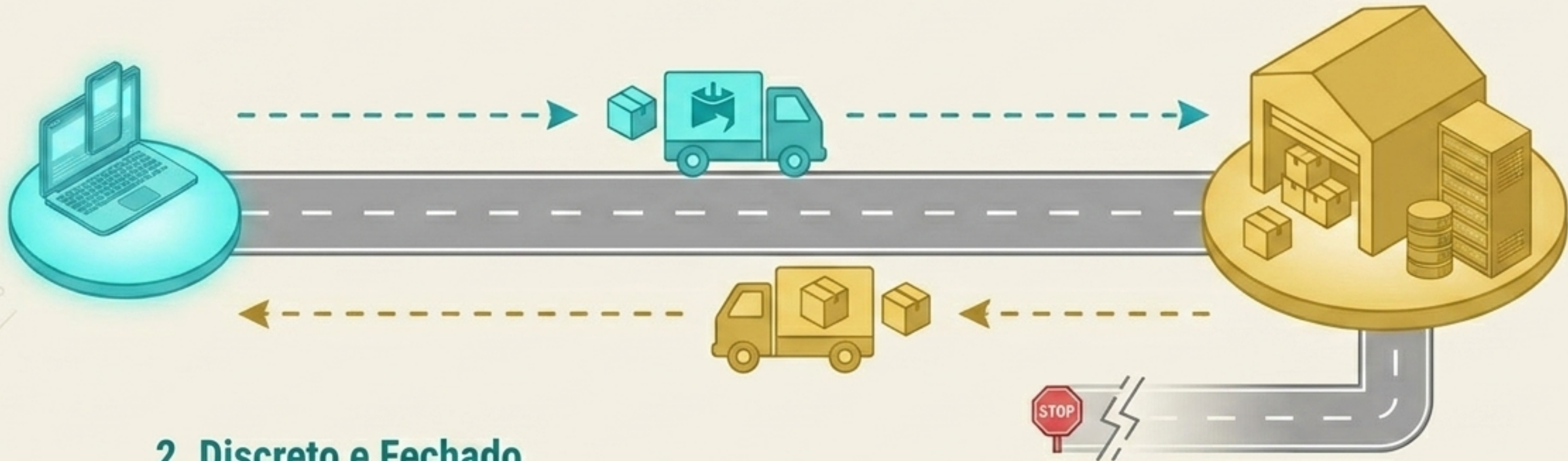
# HTTP: O serviço de entregas discreto

## 1. O Protocolo Padrão

HTTP (HyperText Transfer Protocol) é a espinha dorsal da web.

## 3. Ação Específica

O cliente deve declarar exatamente o que deseja fazer com o recurso usando Verbos.



## 2. Discreto e Fechado

Cada interação é independente. O cliente pede, o servidor responde e a conexão é imediatamente encerrada. Não há comunicação contínua por padrão.



# O vocabulário operacional (Verbos HTTP)

## GET

Seguro



**Propósito:** Solicitar dados.

**Uso:** Carregar páginas ou imagens. Não altera o estado do recurso.

## POST

Não Seguro



**Propósito:** Enviar dados para criar recursos.

**Uso:** Enviar formulários, uploads. Modifica o estado do servidor.

## PUT

Idempotente



**Propósito:** Atualizar um recurso existente ou criar um novo de forma completa.

**Uso:** Atualizar perfis.

## DELETE

Não Seguro



**Propósito:** Remover um recurso especificado.

**Uso:** Deletar registros ou arquivos.



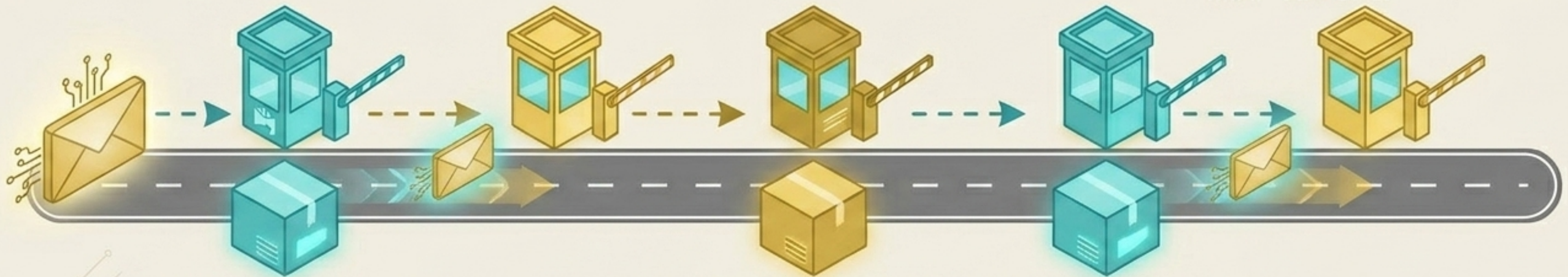
# Estudo de caso: A logística de um E-mail

## 2. Handshake SMTP

Ocorre a saudação entre cliente e servidor, definindo remetente e destinatário.

## 5. Entrega Final

O e-mail repousa no servidor de destino, aguardando resgate via protocolos como IMAP ou POP3.



## 1. Conexão Inicial (MUA)

O cliente (ex: Outlook) conecta-se ao servidor do provedor.

## 3. Transmissão

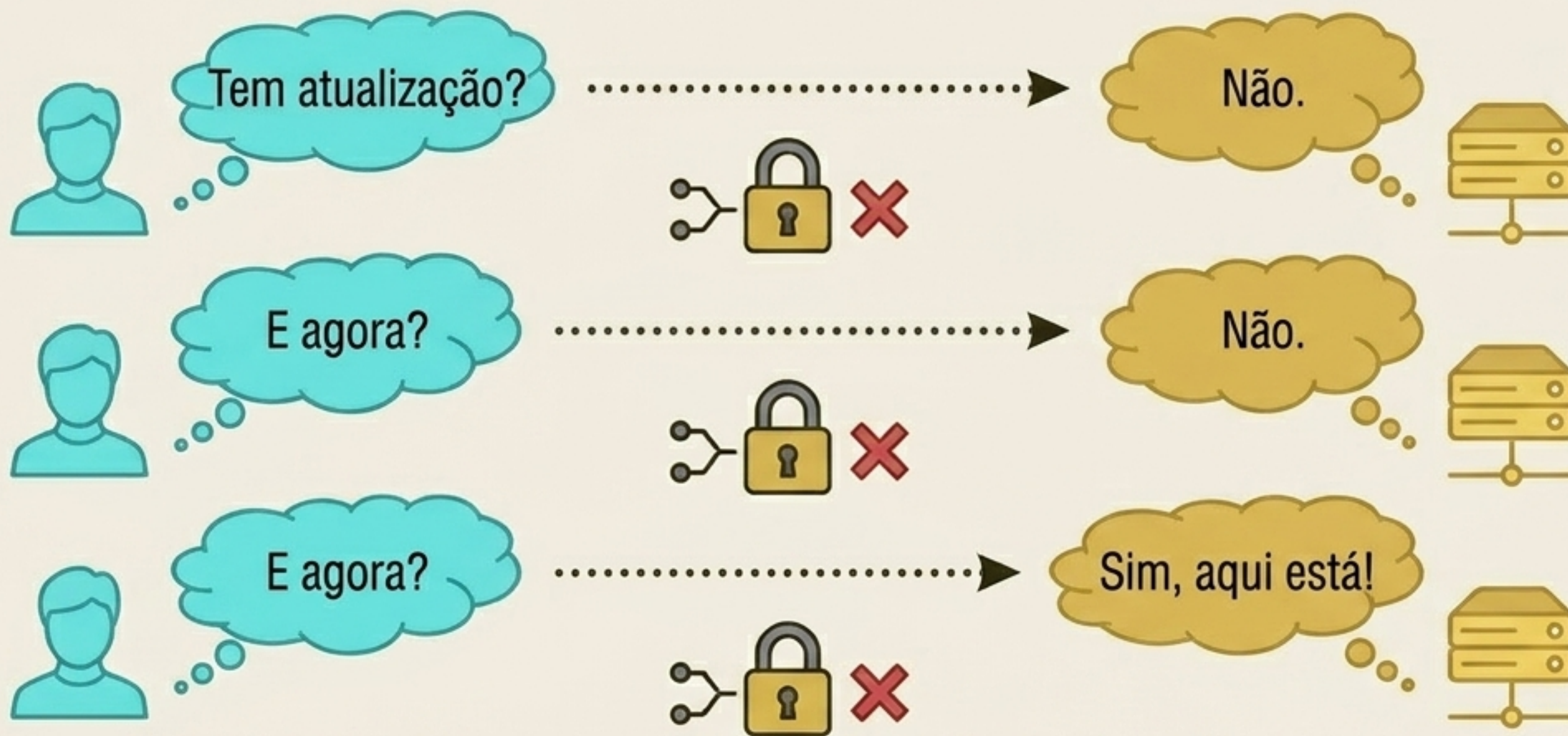
O SMTP trafega o corpo da mensagem e os anexos até o servidor.

## 4. Encaminhamento (DNS)

Se o destino for externo, o servidor consulta o DNS e roteia a mensagem até o servidor final.



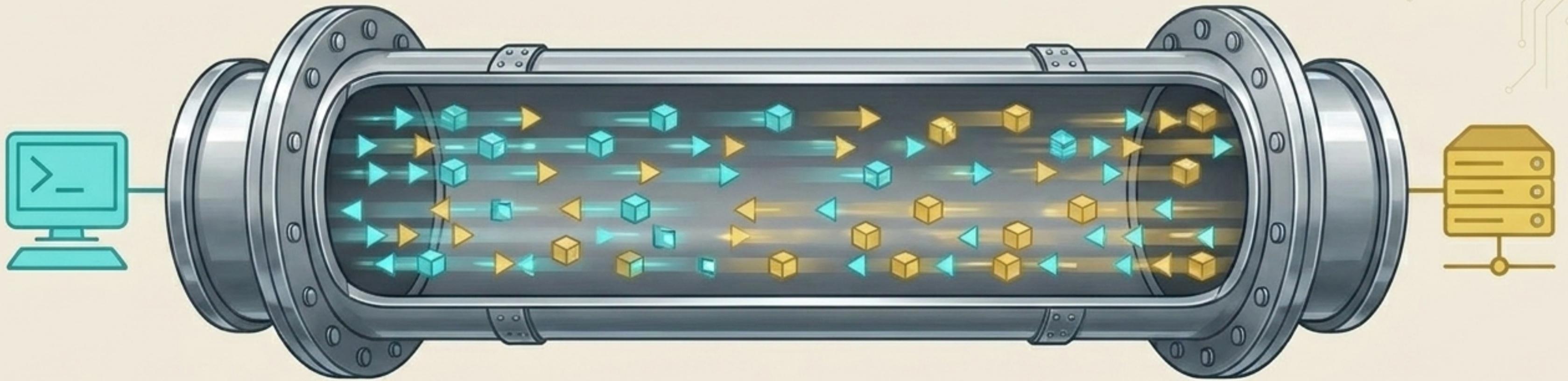
# O gargalo do modelo discreto (Polling)



No modelo tradicional (HTTP), o servidor não pode enviar dados ao cliente de forma proativa. O cliente precisa perguntar constantemente (polling). Para sistemas em tempo real, isso gera latência severa e desperdício massivo de recursos de rede.



# WebSocket: A via expressa contínua



## Comunicação Full-Duplex

Diferente do HTTP, o WebSocket cria um canal bidirecional baseado em TCP.



## Aberto e Persistente

A conexão permanece ativa. O cliente não precisa mais perguntar; o servidor envia os dados no exato milissegundo em que eles estão disponíveis.

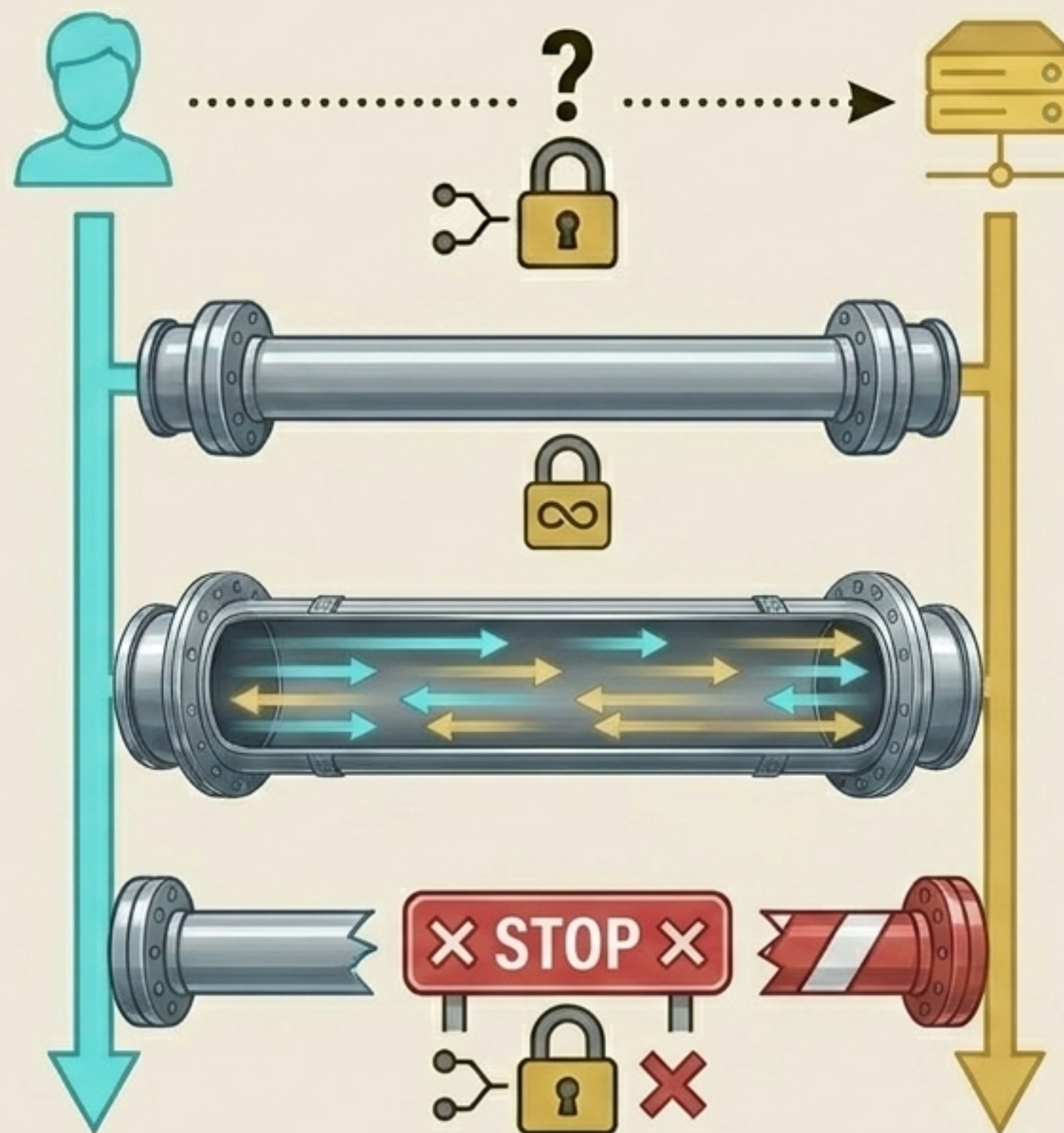


## O Fim da Latência

Elimina a necessidade de abrir e fechar conexões repetidamente.



# Anatomia de uma conexão contínua



## 1. O Handshake (Via HTTP)

O cliente inicia a conexão usando o HTTP tradicional, mas solicita um upgrade para o protocolo WebSocket.

## 2. Conexão Estabelecida

O servidor aceita. O canal bidirecional é aberto com sucesso.

## 3. Fluxo em Tempo Real

Cliente e servidor enviam e recebem mensagens livremente, em qualquer direção, sem novas requisições.

## 4. Fechamento Explícito

A rodovia permanece aberta até que uma das partes decida ativamente encerrar a conexão.



# Quando o tempo real é inegociável



**Chat em tempo real**  
(WhatsApp Web, Slack):  
Mensagens entregues e recebidas instantaneamente sem recarregar a tela.



**Streaming Financeiro**  
Cotações de ações e criptomoedas em bolsas de valores, onde milissegundos equivalem a lucro ou prejuízo.



**Multijogador Online**  
Jogos que exigem sincronia perfeita de posições e ações de dezenas de jogadores simultâneos.



**IoT (Internet das Coisas)**  
Sensores industriais ou dispositivos inteligentes enviando telemetria e dados contínuos para monitores centrais.



# Matriz de Diagnóstico: Qual protocolo escolher?

| Dimensão                 | HTTP                                          | WebSocket                                 |
|--------------------------|-----------------------------------------------|-------------------------------------------|
| Modelo de Comunicação    | Unidirecional<br>(Requisição-Resposta)        | Bidirecional (Full-duplex)                |
| Estado da Conexão        | Discreta / Fecha após a resposta              | Contínua / Aberta e persistente           |
| Iniciativa de Envio      | Apenas o Cliente pode pedir dados             | Cliente ou Servidor podem enviar dados    |
| Custo de Rede (Overhead) | Alto (Handshakes repetitivos)                 | Muito Baixo (Apenas um handshake inicial) |
| Caso de Uso Ideal        | Carregar páginas, CRUDs, envio de formulários | Chats, dashboards ao vivo, games, IoT     |



# A harmonia das arquiteturas



**A arquitetura cliente-servidor não é uma solução de tamanho único.** O domínio da web moderna exige a precisão discreta do **HTTP** para o transporte estruturado de dados e a fluidez contínua do **WebSocket** para a vida em tempo real.

**A arquitetura dita a capacidade.**